

Reinforcement learning based adaptive traffic signal control method invariant to the configuration of the traffic lights

Alexander Yumaganov
Samara National Research University
Samara, Russia
yumagan@gmail.com

Anton Agafonov
Samara National Research University
Samara, Russia
ant.agafonov@gmail.com

Vladislav Myasnikov
Samara National Research University
Samara, Russia
vmyas@geosamara.ru

Abstract—The paper presents an adaptive traffic signal control method that is invariant to its configuration. The proposed method uses a single neural network model to control traffic lights of different configurations, i.e. having a different set of phases and number of controlled lanes. Experimental studies on the effectiveness of the proposed method were carried out in a specialized traffic flow modeling environment, SUMO (Simulation of Urban MObility). The results obtained confirmed the operability and efficiency of the proposed method.

Keywords— traffic signal, adaptive control, reinforcement learning

I. INTRODUCTION

The efficient management of road traffic remains a pressing issue in modern society due to the annual increase in the number of vehicles on the road. Major cities face significant traffic congestion, resulting in longer travel times, higher fuel consumption, and increased air pollution. These factors have a detrimental impact on the overall cost and quality of transportation services. One of the most cost-effective solutions for reducing congestion is the use of traffic lights at intersections. Regulating the flow of vehicles can improve overall traffic efficiency. Intelligent traffic signal control systems can further enhance the effectiveness of traffic management, ensuring smoother and more efficient traffic flow. The use of traffic lights as a tool for traffic management is a practical and effective solution for addressing the challenges posed by increasing traffic congestion in urban areas.

The standard traffic signal control method is based on the predefined schedule for changing operating phases of traffic signal and does not take into account the current state of traffic. Adaptive control methods, on the contrary, select the phase of traffic light regulation based on data on the movement of vehicles in the neighborhood of the controlled intersection. The classic deterministic traffic signals control method [1], using information about the volume of traffic flow, controls traffic light signals by regulating the duration of the entire traffic signal cycle and, accordingly, the duration of each individual phase of the traffic signal cycle. Another adaptive control method, MAXPWFlow [2, 3], controls traffic flow at an intersection by selecting the next phase of a traffic signal based on the predicted value of traffic flow. The forecast is carried out based on the information about vehicles movement characteristics in the incoming traffic lanes adjacent to the controlled intersection.

With the growth of connected vehicles capable of exchanging information about traffic parameters with transport infrastructure, and the development of traffic monitoring systems, the amount of data that can be used to effectively solve different transportation problems has significantly increased. Such problems include traffic flow prediction [4], motion control of autonomous vehicles [5], vehicle collision avoidance [6], and traffic control at a regulated intersection [7]. Recently, there has been a growing interest in adaptive traffic signal control methods that use a machine learning approach. These methods employ reinforcement learning (RL) algorithms to effectively solve the problem of controlling traffic flow at an intersection. The IDQN (Independent Deep Q-Network) method [7] uses the deep Q-learning method to train independent agents, each of which regulates traffic at a separate intersection. To describe the state of each agent, the following values are used: the index of the traffic signal present phase, number of cars in each of the inbound lanes and the sum of their speeds, and the number of idle cars in each of the inbound lanes. The CoLight method [8] uses graph neural networks, which allow to exchange the traffic information between the nearest controlled intersections. The work [9] presents an adaptive traffic signal control method, which takes into account the presence of unconnected cars in the transport network and uses a modified version of the A2C reinforcement learning method.

In many cases, reinforcement learning based adaptive control methods are superior to deterministic adaptive methods, but they have a significant drawback. If it is necessary to add a new traffic signal to the transport network, the corresponding agent must be pre-trained. At the same time, in most cases, it is not possible to use previously trained agents, since the configuration of the added traffic signal may differ from the configuration of existing traffic signals with already trained agents. The proposed in this work adaptive traffic signal control method is free of this drawback.

II. PROBLEM STATEMENT

The problem of controlling traffic signals at an intersection using reinforcement learning methods (RL methods) is usually represented as a Markov decision process, which can be described by a tuple $\langle S, A, P, R, \gamma \rangle$ [10], where S is the state space of the environment, A is the action space, P is the probabilistic state transition function, R is the reward function, γ is the discount factor.

The agent observes the environment state $s_t \in S$ and then takes an action $a_t \in A$ at time step t . As a result of this action, the state of the environment changes in accordance

The work was supported by the Russian Science Foundation grant No. 21-11-00321, <https://rscf.ru/en/project/21-11-00321/>

with the function $P(a_t, s_t; s_{t+1}) = \Pr(s_{t+1} | s_t, a_t)$. $R(s_t, a_t; s_{t+1})$ this is an immediate reward received by the agent upon transition from the state s_t to the state s_{t+1} by performing action a_t . The agent's goal is to find a policy $\pi^*: S \rightarrow A$, that maximizes the expected return, defined as follows:

$$G_t = \sum_{i=0}^T \gamma^i r_{t+i},$$

where T is the total number of time steps, $\gamma \in [0, 1]$ is a discount factor that represents the importance of future rewards in comparison to immediate rewards in determining overall value.

One way to solve the problem of finding an optimal policy π^* is based on finding the optimal action-value function $Q^*(s, a)$ (Q-learning). The action-value function $Q^\pi: S \times A \rightarrow \mathbb{R}$ is defined as follows:

$$Q^\pi(s, a) = E[G_t | s_t = s, a_t = a].$$

The Bellman equation is used to find the optimal action-value function $Q^*(s, a)$:

$$Q^*(s_t, a_t) = E \left[R(s_t, a_t; s_{t+1}) + \gamma \max_{a_{t+1}} Q^*(s_{t+1}, a_{t+1}) \right] \quad (1)$$

Therefore, the optimal policy is determined as follows:

$$\pi^*(s) = \arg \max_a Q^*(s, a).$$

In order to address the problem of adaptive traffic signal control at an intersection using reinforcement learning, it is essential to define the environment state description, determine the agent's action space, establish the reward function, and select the appropriate RL learning method. We consider in turn the RL method used (Section 3.A), the construction of the environment state description (Section 3.B), the reward function used, and the action space (Section 3.C).

III. PROPOSED METHOD

A. Deep Q-Learning from Demonstrations

As a reinforcement learning method, it is proposed to use the Deep Q-Learning from Demonstrations (DQfD) method [11], which uses the experience of an expert during the agent's training process.

The training process for the agent using the DQfD method involves two stages. At the first stage, initial imitation learning of the model is carried out. At this stage, only the data of the "expert" are used as training data. In this work, as expert data, it is proposed to use data obtained using the adaptive deterministic traffic signals control method based on a prediction of maximum weighted flow (MaxPWFlow) [3]. The loss function utilized in the training process has the following form:

$$J(Q) = J_{DQ}(Q) + \lambda_1 J_n(Q) + \lambda_2 J_E(Q) + \lambda_3 J_{L2}(Q) \quad (2)$$

where $J_{DQ}(Q)$ is the Double DQN loss function, $J_n(Q)$ is an n-step Double DQN loss function, $J_E(Q)$ is a supervised large margin classification loss, $J_{L2}(Q)$ is a L2 regularization loss, $\lambda_1, \lambda_2, \lambda_3$ are weight coefficients.

The loss function $J_E(Q)$ is used only for data received from an expert; otherwise, the corresponding coefficient λ_2 is set to zero. Supervised large margin classification loss $J_E(Q)$ is defined as follows:

$$J_E(Q) = \max_{a \in A} [Q(s, a, \theta) + l(a_E, a)] - Q(s, a_E, \theta),$$

where a_E is the action taken by an expert in state S , $l(a_E, a)$ is a margin function, which is defined as follows:

$$l(a_E, a) = \begin{cases} 0, & \text{if } a_E = a \\ m > 0, & \text{else} \end{cases}.$$

The loss function $J_E(Q)$ is constructed so that the Q-values of actions that were not selected by the expert are always lower by value no less than m to the Q-values of the expert's action in the same state. If only the $J_E(Q)$ function is used as a loss function at the first stage of training, the Q-values returned by the trained neural network will not satisfy the Bellman equation, on the basis of which Q-learning is based (1). Therefore, Q-learning loss functions ($J_{DQ}(Q)$ and $J_n(Q)$) are also used at all stages of training.

The DQfD method is based on the Double DQN Q-learning method [12]. During the training process, this method uses two neural networks to approximate the values of the Q-function: the target network and the current one. The same neural network architecture is used in the target and current networks, but its weight coefficients are formed by copying the corresponding weight coefficients of the current network at every τ training steps. The target neural network is used to evaluate the action selected by the current network. In order to solve the problem of overestimating the agent's actions, two separate neural networks need to be used. The loss function for the Double DQN method has the following form:

$$J_{DQ}(Q) = (r + \gamma Q(s_{t+1}, \arg \max_a Q(s_{t+1}, a, \theta), \theta_r) - Q(s_t, a, \theta))^2,$$

where θ and θ_r are the weight coefficients of current and target networks respectively. Then the n-step Double DQN loss function is defined as follows:

$$J_n(Q) = \left(\sum_{i=0}^{n-1} \gamma^i r_{t+i} + \gamma^n Q(s_{t+n}, \arg \max_a Q(s_{t+n}, a, \theta), \theta_r) - Q(s_t, a, \theta) \right)^2.$$

The neural network architecture that approximates the Q-function values used in the proposed method is shown in Figure 1.

At the second stage, online training of the model is carried out based on the Double DQN reinforcement learning method. The loss function remains unchanged in the second stage. During the training process, the agent engages with the environment and generates a set of data that are added to the special storage called the replay buffer. The contents of the replay buffer are used as training data in both stages of DQfD method. At the beginning of the second stage, the replay buffer contains only expert data. The replay buffer has a limited size and when it is full, old data are overwritten. But only the data of the trained agent is overwritten in such cases, and the data received by the expert are not affected. The authors of DQfD use Prioritized Experience Replay [13] as a replay buffer, in which the probability of selecting data for training is directly proportional to the training error on these data.

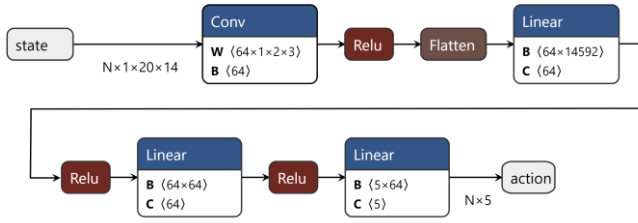


Fig. 1. Neural network architecture.

B. Environment state representation

The state of a controlled intersection is proposed to be described using a feature vector that characterizes each pair of traffic lanes (incoming lane and corresponding outgoing lane) at the intersection. Moreover, one incoming lane can correspond to several outgoing lanes and vice versa. The feature vector for one such pair of traffic lanes can be divided into two parts: dynamic and static. The dynamic part of the feature vector characterizes the present state of traffic flow on the corresponding pair of lanes at the intersection. While the static part of the feature vector characterizes the configuration of the controlled intersection and the corresponding traffic signal.

It is proposed to use the following characteristics of the current state of traffic flow in the lane l :

- indicator of the possibility of driving through the intersection in the lane l (only incoming) when the current phase φ of the traffic signal is active:

$$I_l(\varphi) = \begin{cases} 1, & \text{if driving is allowed} \\ 0, & \text{else} \end{cases}$$

- number of moving vehicles: MV_l ;
- number of halting vehicles: HV_l ;
- sum of vehicles waiting time: WT_l ;
- average speed of vehicles: AS_l .

Thus, the dynamic part of the feature vector for one pair of inbound in and outbound out lanes at an intersection has the following form:

$$(I_{in}(\varphi), MV_{in}, HV_{in}, WT_{in}, AS_{in}, MV_{out}, HV_{out}, WT_{out}, AS_{out}).$$

The static part of the feature vector allows to use one model of agent to control traffic signals with a different set of phases at different types of intersections. For each pair of incoming-outgoing intersection lanes (in and out), a vector is formed (the static part of the feature vector), the dimension of which is equal to the maximum number of main phases of a traffic signal. The main phase refers to those phases of a traffic signal that do not contain transition signals ("yellow" signals). For a phase with order number n_φ , the value of the corresponding element of the static part of the feature vector is determined as follows:

$$S_{in}^{out}(n_\varphi) = \begin{cases} 0, & \text{if there is no such phase index} \\ 1, & \text{if vehicle movement is not allowed} \\ 2, & \text{else} \end{cases}$$

In this work, the maximum number of main phases of traffic signal is set to five. By combining the corresponding dynamic and static parts of the feature vector, we obtain the final feature vector for a pair of incoming and outgoing lanes:

$$(I_{in}(\varphi), MV_{in}, HV_{in}, WT_{in}, AS_{in}, MV_{out}, HV_{out}, WT_{out}, AS_{out}, S_{in}^{out}(0), S_{in}^{out}(1), S_{in}^{out}(2), S_{in}^{out}(3), S_{in}^{out}(4))$$

The maximum number of such pairs in this work is assumed to be 20. If there are fewer number of pairs, the final feature vector is padded with zeros. Thus, the final dimension of the feature vector describing the state of the environment (controlled intersection) in this work is 280.

C. Reward function

The value of the reward is proposed to be calculated based on the value of the intersection's pressure, which is defined as the difference of the number of vehicles approaching the intersection and the number of vehicles moving away from the intersection along the corresponding entry and exit lanes. Thus, it is proposed to use the following function as a reward function:

$$R(s_t, a_t; s_{t+1}) = -P_t^{t+1}$$

where P_t^{t+1} is the "pressure" of the intersection for the time interval $(t, t+1)$.

D. Action space

The action space includes indexes of the main phases of the traffic signal. The size of the action space is equal to the maximum number of main phases in the controlled traffic signal. As stated above, in this work, the maximum number of main phases of a traffic signal is assumed to be five. However, traffic signals may have a smaller number of main phases than the established maximum number. As a result, a situation may arise in which the RL method returns a phase index that exceeds the number of main phases of the traffic signal in question. To overcome this issue, it is suggested to select the index of the traffic signal phase as the remainder of dividing the phase index obtained using the RL method by the maximum value of the phase index of the traffic signal under consideration.

IV. EXPERIMENTS

The effectiveness of the proposed method was experimentally investigated using a microscopic traffic simulation system SUMO. (Simulation of Urban MObility) [14]. The suggested method was compared compared to the following methods: the deterministic adaptive traffic signal control method MaxPWFlow, which was used as an “expert”; the deep Q-learning based method IDQN (Independent Deep Q-Network) [7], in which one agent controls traffic flow on one intersection; the classic traffic signals control method FixedTime, in which the timing of the signal phases is set in advance and does not adapt to current traffic conditions. Traffic light control methods were compared based on the following criteria: average travel time of vehicles, and average waiting time of vehicles.

Experimental studies were conducted on three simulation scenarios constructed on the basis of the real city scenario “TAPAS Cologne” [15]:

- Scenario “cologne8”. This scenario represents a segment of Cologne's transportation system, featuring 8 traffic signals controlled intersections of different designs. The number of vehicles in the scenario is 4214.
- Scenario “cologne3”. This scenario represents a short section of the highway of Cologne's transportation system and contains 3 intersections controlled by traffic signals of different structures. The number of vehicles in the scenario is 2856.
- Scenario “cologne1”. The scenario contains one controlled intersection. The number of vehicles in the scenario is 2014.

Figure 2 shows the transport networks used in the above mentioned scenarios.

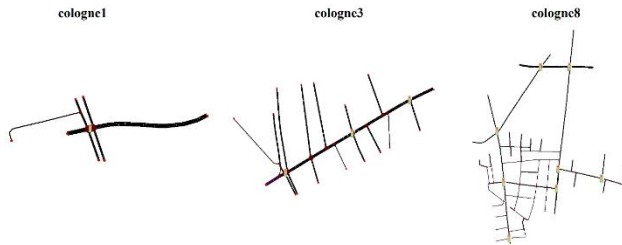


Fig. 2. Transport system networks of the simulation scenarios used in experimental studies.

The experiments were run with a simulation interval of 1 second. The overall simulation time was 3600 seconds. All models were compared on the same data - a sample of ten episodes. The episodes differed from each other in the value of the “seed” parameter of the SUMO system, which affects the starting positions of the vehicles on the segments of the transport network and the dynamics of their movement 5.

During the initial phase of the experiments, the convergence of the proposed method was evaluated. The learning curve of the proposed method on the “cologne3” scenario is shown in Figure 3. According to the results obtained, the method was successfully trained and achieved stable total reward values.

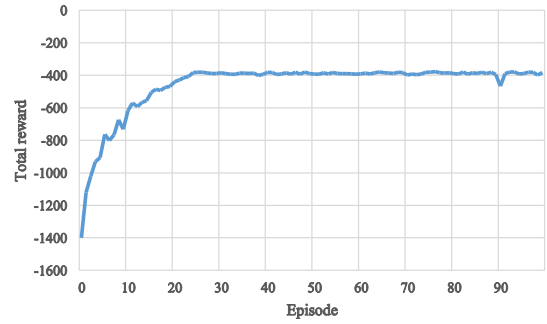


Fig. 3. Convergence of the proposed method on “cologne3” scenario.

In the following experimental stage, the method presented in this paper was compared with the adaptive deterministic traffic signal control method MaxPWFlow, which was used as an “expert”. The scenarios used were “cologne8” and “cologne3”. The model was first trained on the “cologne8” scenario. Since the configuration of the traffic signals in the “cologne3” scenario is unique compared to the traffic signals in the “cologne8” scenario, the previously used neural network model was additionally trained on this scenario. The results are presented in Table 1. Based on the results obtained, the proposed method outperforms both the “expert” method used for training and the FixedTime method in in each of the scenarios considered.

TABLE I. COMPARISON OF METHODS

Method	Cologne8		Cologne3	
	Average waiting time, s	Average travel time, s	Average waiting time, s	Average travel time, s
Proposed method	14.20	94.63	8.26	58.38
MaxPWFlow	14.75	101.48	8.98	59.22
FixedTime	61.55	152.17	25.06	76.48

TABLE II. COMPARISON OF METHODS ON “NEW” SCENARIO

Method	Cologne1	
	Average waiting time, s	Average travel time, s
Proposed method (trained on cologne3)	11.28	49.58
Proposed method (trained on cologne1)	9.36	46.04
MaxPWFlow	9.57	47.15
IDQN (trained on cologne3)	33.10	70.84
IDQN (trained on cologne1)	9.42	45.24
FixedTime	30.36	67.50

During the final stage of the experimental study, the possibility of applying the model to a new scenario not involved in the training process was investigated. The “cologne1” scenario was used as such a scenario. For this study, two models of the proposed method were used - the model trained on the “cologne3” scenario and the model additionally trained on the cologne1 scenario. Comparisons were made with two similarly trained models of the IDQN RL method: the model trained on the “cologne3” scenario and the model trained on the cologne1 scenario. The results are

presented in Table 2. The results show that the proposed method can be used on a “new” scenario without additional training on it, unlike the IDQN method. After additional training on the scenario under test, the proposed method shows the best result in average waiting time and the second highest results in average travel time.

V. CONCLUSION

This paper presents an adaptive traffic signal control method that is invariant to the configuration of the traffic lights. The proposed method is based on the application of the reinforcement learning method DQfD, which additionally uses the experience of an “expert” in the training process. The proposed method of forming a vector representation of the environment state allows to use a single agent model for controlling traffic signals of different types. The results of the conducted experimental studies have shown the effectiveness of the proposed method.

Further research will concentrate on studying the effectiveness of the proposed method on a large urban traffic network, which includes a large number of traffic signals of different types. Another direction of further research is to study the effectiveness of different “expert” methods of adaptive traffic signal control to determine which approach is the most effective in managing traffic flow at intersections.

REFERENCES

- [1] F. V. Webster, “Traffic Signal Settings,” Road Research Technique Paper No. 39, Road Research Laboratory, London, 1958.
- [2] A. Agafonov, A. Yumaganov, and V. Myasnikov, “Adaptive traffic signal control based on maximum weighted traffic flow,” in 2022 VIII International Conference on Information Technology and Nanotechnology (ITNT), 2022, pp. 1-6.
- [3] V. V. Myasnikov, A. A. Agafonov, and A. S. Yumaganov, “A deterministic predictive traffic signal control model in intelligent transportation and geoinformation systems,” *Computer Optics*, vol. 45, no. 6, pp. 917–925, 2021, doi: 10.18287/2412-6179-CO-1031.
- [4] L. L. Chen, “Research on robot motion control and trajectory tracking based on agricultural seeding,” *Computer Optics*, vol. 47, 2023.
- [5] J. Pamplona, C. Madrigal, and J. Herrera-Ramirez, “Transformer point net: cost-efficient classification of on-road objects captured by light ranging sensors on low-resolution conditions,” *Computer Optics*, vol. 46, no. 2, pp. 326–334, Apr. 2022, doi: 10.18287/2412-6179-CO-1001.
- [6] A. A. Agafonov, A. S. Yumaganov, and V. V. Myasnikov, “Big data analysis in a geoinformatic problem of short-term traffic flow forecasting based on a K nearest neighbors method,” *Computer Optics*, vol. 42, no. 6, pp. 1101–1111, 2018.
- [7] J. Ault, J. P. Hanna, and G. Sharon, “Learning an interpretable traffic signal control policy,” *arXiv [cs.LG]*, 2019.
- [8] H. Wei et al., “CoLight: Learning network-level cooperation for traffic signal control,” in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, 2019, pp. 1913–1922.
- [9] Z. Mo, W. Li, Y. Fu, K. Ruan, and X. Di, “CVLight: Decentralized learning for adaptive traffic signal control with connected vehicles,” *Transportation Research Part C: Emerging Technologies*, vol. 141, no. 103728, p. 103728, 2022.
- [10] H. Wei, G. Zheng, V. Gayah, and Z. Li, “A survey on traffic signal control methods,” *arXiv [cs.LG]*, 2019.
- [11] T. Hester et al., “Deep Q-learning from demonstrations,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [12] H. Van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with Double Q-learning,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016.
- [13] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” *arXiv [cs.LG]*, 2015.
- [14] D. Krajzewicz, J. Erdmann, M. Behrisch, and L. Bieker, “Recent Development and Applications of SUMO - Simulation of Urban MObility,” *International Journal on Advances in Systems and Measurement*, vol. 5, no. 3&4, pp. 128–138, Dec. 2012.
- [15] “TAPASCologne - SUMO documentation,” Dlr.de. [Online]. Available: <https://sumo.dlr.de/docs/Data/Scenarios/TAPASCologne.html>. [Accessed: 5-Dec-2023].